Symplichain Technologies

Software Engineering Intern — Hackathon Submission

Submitted to:	ashit.swain@symplichain.com
Role:	Software Engineering Intern
Date:	April 2026
GitHub:	Workflows in .github/workflows/ directory

Part 1 (40%)	Shared Gateway Problem — Rate Limiting & Fair Queue
Part 2 (20%)	Mobile App Architecture
Part 3 (20%)	CI/CD and Deployment Pipeline + GitHub Actions YAML
Part 4 (20%)	Debugging — The Monday Morning Outage

Part 1: Shared Gateway Problem (40%)

Overview

25 customers collectively generate >20 req/s during peak hours against an external API capped at 3 req/s. The core challenge is dual: enforce the hard rate limit without null responses, and ensure fair access across customers so no single customer can starve others. The solution reuses existing infrastructure (Redis + Celery) with no new AWS services needed.

Architecture Diagram

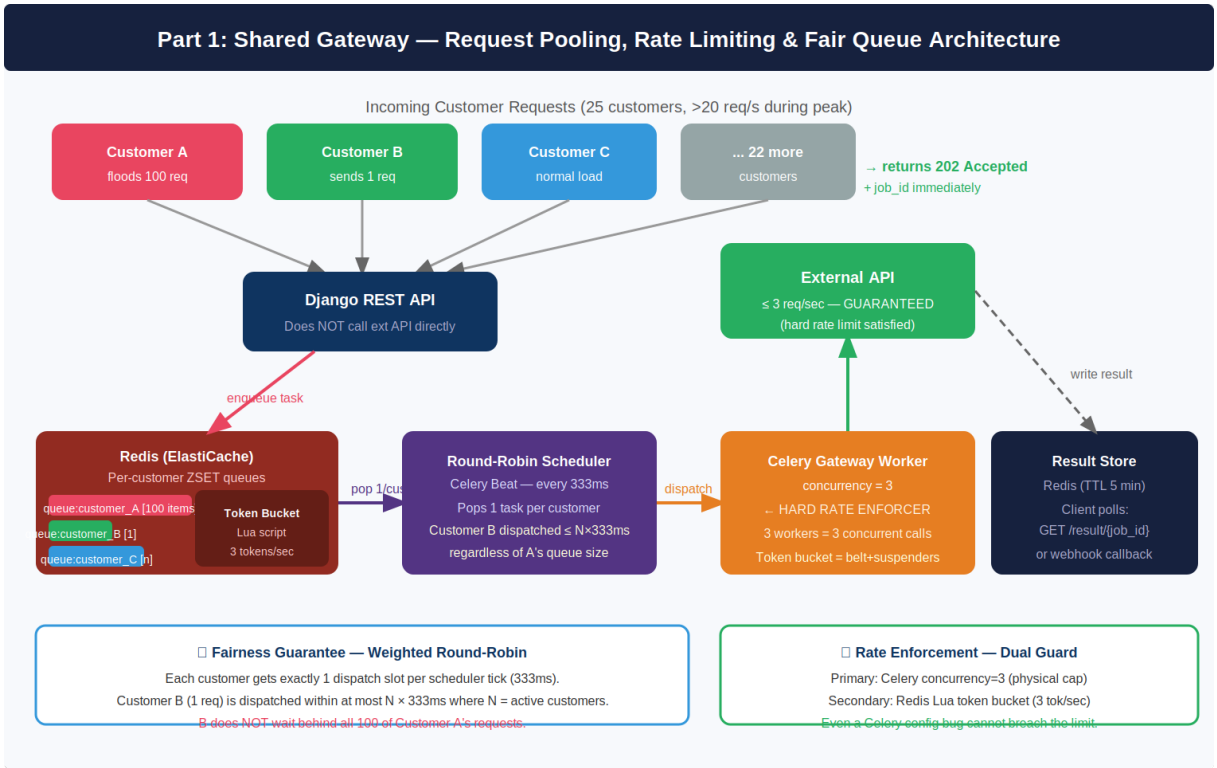

Figure 1: Gateway rate-limiting architecture — per-customer Redis queues, round-robin scheduler, Celery concurrency=3 as hard rate enforcer

1.1 Components

- **Django API (no change to request path):** The inbound view enqueues the task into a Redis sorted-set (ZSET) keyed by customer ID, scored by timestamp, then immediately returns HTTP 202 Accepted with a job_id. The caller never blocks waiting for the external API.
- **Redis per-customer ZSET queues:** Each customer has symplichain:queue:{customer_id} as a sorted set. Tasks are pushed with ZADD; the scheduler pops with ZPOPMIN. This gives O(log N) enqueue/dequeue and natural FIFO ordering per customer.
- **Token Bucket (Redis Lua):** A Lua script runs atomically to manage a 3-token-per-second bucket stored in Redis. The scheduler acquires a token before dispatching. If the bucket is empty it waits for the next 333ms tick. This is the belt-and-suspenders backup to Celery concurrency.
- **Round-Robin Scheduler (Celery Beat, every 333ms):** Maintains an ordered list of active customers (those with >0 queued tasks). On each tick, advances a pointer and dispatches one task from that customer's queue to the gateway_queue. This is the fairness mechanism.
- **Celery Gateway Worker (concurrency=3):** A dedicated Celery queue with concurrency=3 is the primary rate enforcer. 3 workers = maximum 3 concurrent HTTP calls to the external API. Each worker holds its slot for the duration of the call (~100-300ms).

• **Result Store:** Results are written to Redis with TTL=5min. The original caller polls GET /api/gateway/result/{job_id}/ or receives a webhook callback. The original request path returns instantly, so the caller never hangs.

1.2 Rate Enforcement — Exactly ≤ 3 req/s

Two independent guards prevent exceeding the rate limit:

Primary: Celery concurrency=3 physically caps concurrent calls. At 100-300ms per external call, throughput is <3 req/s by construction.

Secondary: Redis Lua token bucket refills at exactly 3 tokens/second. The scheduler acquires a token before each dispatch. Even a Celery misconfiguration cannot breach the limit because the token bucket operates independently.

Token Bucket Lua Script (runs atomically on Redis):

```
-- Lua token bucket — called by scheduler before each dispatch
local key    = KEYS[1]               -- "symplichain:gateway:tokens"
local limit  = tonumber(ARGV[1])     -- 3
local now    = tonumber(ARGV[2])     -- epoch ms
local refill = tonumber(ARGV[3])     -- 333 (ms per token)

local last   = tonumber(redis.call('HGET', key, 'last') or now)
local tokens = tonumber(redis.call('HGET', key, 'tokens') or limit)

-- refill proportionally to elapsed time
local elapsed    = now - last
local new_tokens = math.min(limit, tokens + math.floor(elapsed / refill))

if new_tokens >= 1 then
    redis.call('HSET', key, 'tokens', new_tokens - 1, 'last', now)
    return 1   -- token acquired, dispatch allowed
else
    return 0   -- denied, scheduler waits for next tick
end
```

1.3 Fairness — Weighted Round-Robin

If Customer A floods 100 requests and Customer B sends 1, a naive FIFO queue would make B wait behind all 100 of A's tasks. The round-robin scheduler prevents this:

The scheduler maintains a list of active customers (those with >0 queued items). On each 333ms tick it advances a pointer and dispatches exactly one task from the current customer's queue. The result: if there are N active customers, Customer B's task is dispatched within at most N × 333ms — regardless of Customer A's queue depth. With 25 customers that is at most ~8 seconds worst case, not the minutes a pure FIFO would cause.

Optional extension: For paid tiers, weight the round-robin. Premium customers get 2 slots per round, standard get 1. Implemented by storing weights in a Redis hash and repeating their queue index in the rotation list accordingly.

1.4 Failure Handling & Retry Strategy

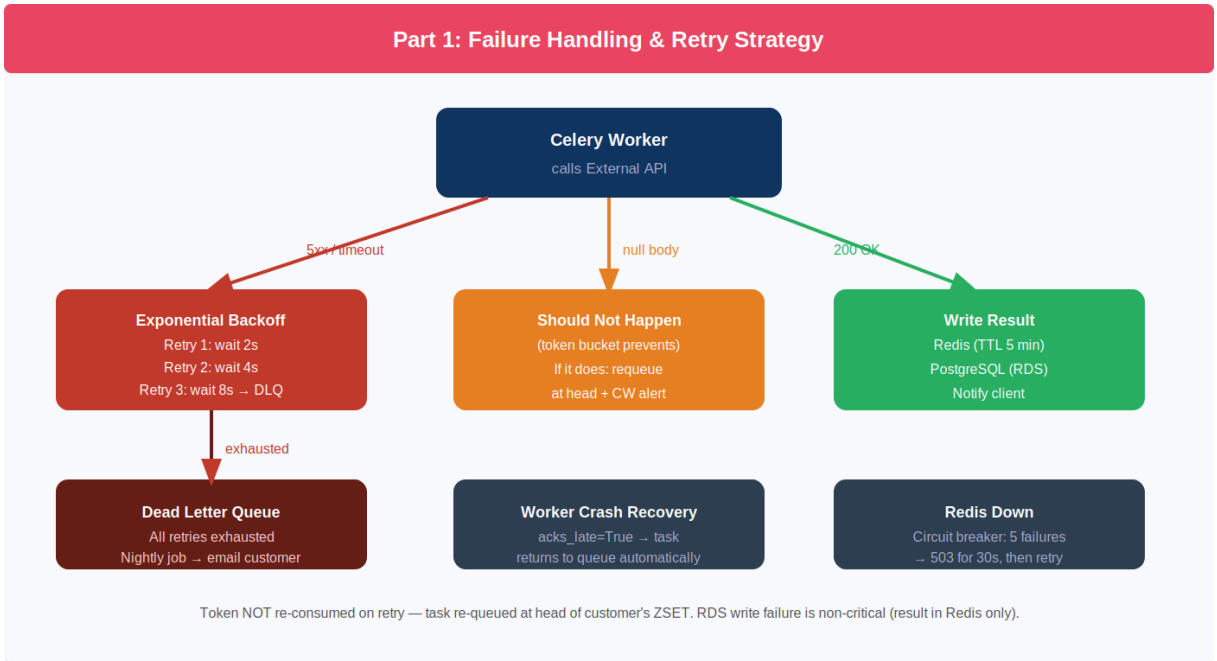

Figure 2: Failure handling flow — exponential backoff, dead-letter queue, circuit breaker

Scenario	Detection	Strategy
External API 5xx / timeout	HTTP status ≥500 or requests.Timeout	Exponential backoff: retry after 2s, 4s, 8s (max 3 retries). Token NOT re-consumed
Null response (rate limit breach)	Response body is None/empty	Should not occur with dual guard. If it does: requeue at head + emit CloudWatch cu
Celery worker crash	acks_late=True + visibility timeout	Task returns to queue automatically. Systemd unit restarts Celery with Restart=on-
Redis unavailable	ConnectionError in scheduler	Scheduler logs error, pauses 5s, retries. Django API activates circuit breaker: 5 con
All retries exhausted	3 retries failed	Task moves to dead-letter queue (DLQ). Nightly job scans DLQ and notifies custom
RDS write failure (result)	SQLAlchemy/psycopg2 error	Non-critical: write result to Redis only (TTL 10min). Log + CloudWatch alert. Does

Part 2: Mobile App Architecture (20%)

Architecture Diagram

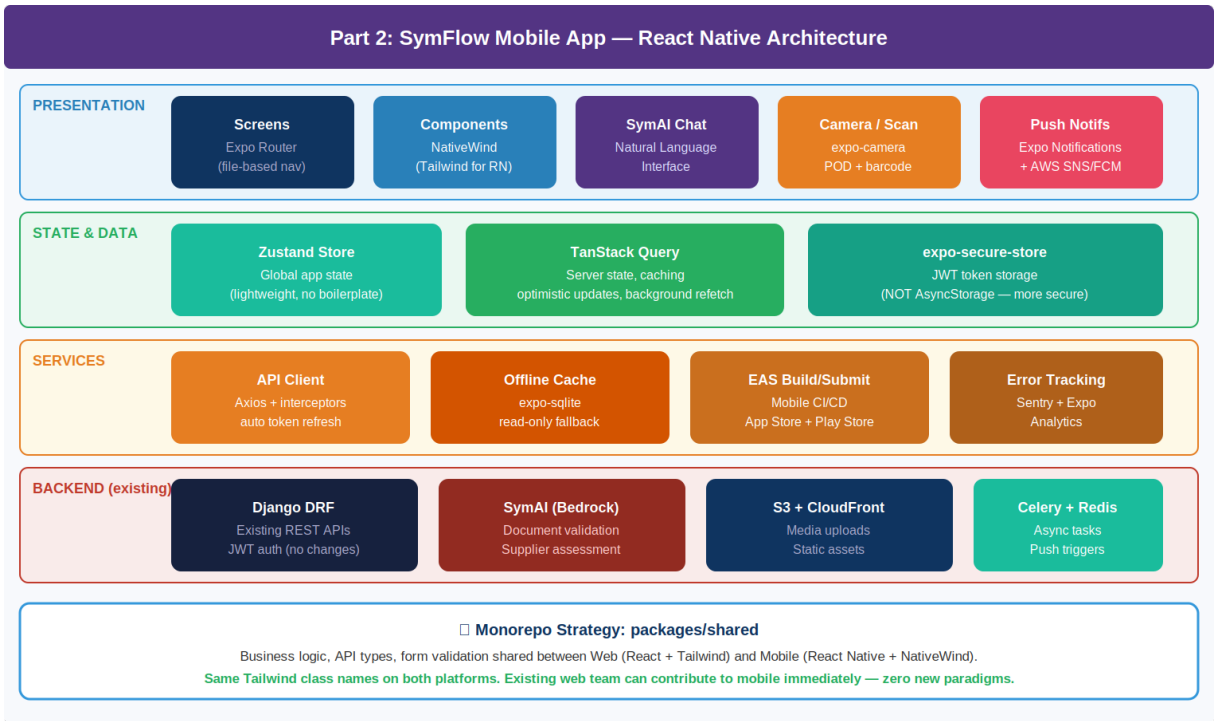

Figure 3: SymFlow mobile app layered architecture — React Native + Expo

2.1 Interaction Model

SymFlow's mobile users are supply chain operators who need to act quickly, often while moving in warehouses or on loading docks. I recommend a **gesture/tap-first with contextual NL assistant** model:

- **Primary: Gesture/tap-driven (80% of tasks):** Core actions — approve shipment, flag exception, view tracking — are reachable in 1–2 taps via large thumb-friendly action cards. No keyboard, no typing for the most frequent workflows.
- **Secondary: Natural language via SymAI chat:** A floating assistant button opens a chat powered by SymAI. Handles complex ad-hoc queries: 'Show delayed shipments from Supplier X this week' without navigating 4 screens. Especially useful for managers and power users.
- **Camera-first for field actions:** POD photo uploads, barcode scans, and document capture use the native camera as a first-class input — zero friction for warehouse and driver staff. Maps directly to existing driver app behavior.
- **Push notifications as entry points:** Critical alerts (shipment delayed, validation failed, approval needed) arrive as rich push notifications with inline action buttons. Users can approve/reject without opening the app.
- **Deliberate exclusion of voice:** Warehouse environments are noisy. Voice commands would be unreliable as a primary interaction. Excluded from v1; can be added as an accessibility option in v2.

2.2 Tech Stack — React Native + Expo

Option	Pros	Cons	Decision
React Native (Expo managed)	• Team already knows React+Tailwind • ~70% logic shared with web • Single codebase iOS+Android • NativeWind = same CSS classes • Fast iteration via Expo Go	• Occasional native module friction • Slightly larger bundle vs native	■ CHOSEN

Native (Kotlin + Swift)	• Best raw performance • Full platform API access	• Two codebases to maintain • No existing mobile engineers • 2× development time	■ Too costly for team size
Flutter (Dart)	• Great performance • Strong widget library	• Team has zero Dart experience • Different design system from web • No code sharing with Django/React	■ Learning curve too steep

Full stack breakdown:

- **Framework:** React Native 0.74 + Expo SDK 51
- **Navigation:** Expo Router (file-based, identical mental model to Next.js — web team gets it immediately)
- **Styling:** NativeWind — Tailwind CSS utility classes work in React Native. Zero new styling paradigm.
- **State:** Zustand for global state (lighter than Redux, no boilerplate), TanStack Query for server state (caching, background refetch, optimistic updates)
- **Auth:** Existing Django JWT. Tokens stored in expo-secure-store (backed by iOS Keychain / Android Keystore — NOT AsyncStorage which is unencrypted)
- **Camera / media:** expo-camera + expo-image-picker for POD uploads and barcode scanning
- **Push notifications:** Expo Notifications + AWS SNS (already in stack) or Firebase FCM
- **Offline:** TanStack Query persistence adapter + expo-sqlite for read-only offline views of recent data
- **CI/CD:** Expo EAS Build + EAS Submit — integrates into GitHub Actions, pushes to App Store + Play Store
- **Shared code:** Monorepo with packages/shared — business logic, API client types, and validation schemas shared between web and mobile, eliminating duplication

Part 3: CI/CD and Deployment Pipeline (20%)

Pipeline Diagram

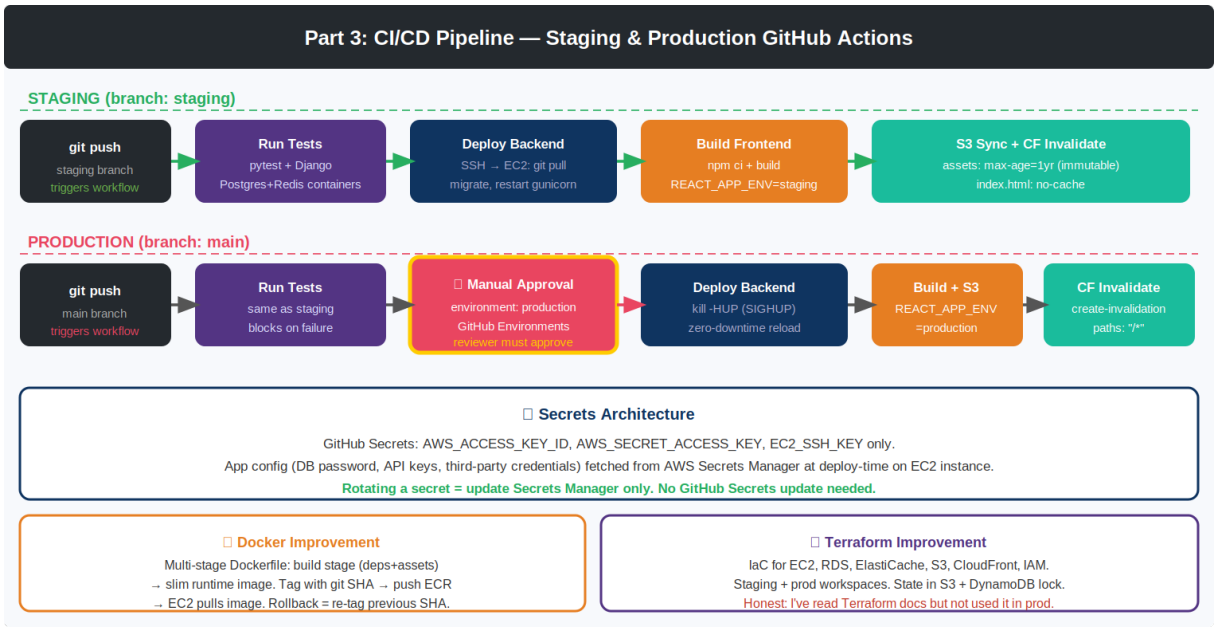

Figure 4: CI/CD pipeline — staging and production tracks with key differences highlighted

3.1 GitHub Actions Workflows

Two complete workflow files are in the repository: **.github/workflows/deploy-staging.yml** and **deploy-production.yml**. Below are the key design decisions.

- **Tests gate both deployments:** Both staging and production run the full Django test suite with real Postgres and Redis service containers before any deploy step. A failing test blocks the deployment completely.
- **AWS Secrets Manager for application secrets:** GitHub Secrets hold only three things: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, and the EC2 SSH PEM key. All application secrets (DB password, third-party API keys, etc.) are fetched from AWS Secrets Manager at runtime on the EC2 instance via a Python one-liner. This means rotating any secret requires only updating Secrets Manager — no GitHub Secrets touch needed.
- **Frontend cache strategy:** JS/CSS/image assets (which have content-hashed filenames from webpack) get Cache-Control: max-age=31536000, immutable (1 year, browser and CDN never re-fetches). index.html gets no-cache, no-store, must-revalidate — the browser always fetches a fresh HTML shell which points to the new hashed asset URLs. Without this, CloudFront can serve stale HTML pointing to JS/CSS files that no longer exist in S3.
- **Production manual approval gate:** The production workflow uses environment: production in GitHub Actions, which requires a named reviewer to approve before the deploy jobs run. This prevents an accidental push to main from automatically deploying to production.
- **Zero-downtime Gunicorn reload on production:** Production uses kill -HUP (SIGHUP to the Gunicorn master PID) which triggers a graceful worker reload: old workers finish in-flight requests before dying, new workers start up and take over. Using systemctl restart would drop all in-flight requests instantly.
- **Staging uses hard restart:** Staging uses systemctl restart gunicorn since it's fine to briefly interrupt staging traffic and the simpler approach is easier to reason about during development.

3.2 Docker Improvement

I'm familiar with Docker. Here's what I would add:

- Multi-stage Dockerfile: Stage 1 installs all Python deps + builds React assets. Stage 2 is a slim runtime image (python:3.11-slim) containing only the production requirements. This cuts image size ~60% and removes build tools from the production environment.
- In CI, the image is built once, tagged with the git SHA (e.g. symplichain:abc1234), and pushed to ECR. The EC2 instance pulls and runs this exact image. Rollback becomes: re-tag the previous SHA image and pull it — no git operations on the server.
- docker-compose.yml for local dev: developer runs docker compose up and gets Django + Celery + Redis + Postgres wired together with no manual virtual environment setup. This eliminates 'works on my machine' issues entirely.
- The GitHub Actions workflow would change from SSH + git pull to: docker build → docker push ECR → SSH → docker pull + restart container.

3.3 Terraform Improvement

Honesty note: I have read the Terraform documentation and understand the concepts (providers, resources, state, workspaces), but I have not written Terraform in production. My approach would be to start with the official AWS provider modules rather than writing everything from scratch. Here is what I would do:

- Codify the current manual AWS setup into .tf files: EC2 instance type and AMI, security groups, RDS PostgreSQL instance, ElastiCache cluster, S3 bucket policies, CloudFront distribution, IAM roles and policies.
- Two Terraform workspaces (staging and production) with separate .tfvars files. Environments are structurally identical but isolated — no risk of a staging operation touching production.
- Remote state stored in an S3 bucket with DynamoDB state locking (standard, cost-effective pattern — avoids concurrent apply conflicts).
- A terraform plan step added to the GitHub Actions workflow that posts the diff as a PR comment, so reviewers can see exactly what infrastructure will change before merging.

Part 4: Debugging — The Monday Morning Outage (20%)

POD photo uploads are failing. Upload path: Driver app → Django API (EC2) → S3 → Celery task → Bedrock model → PostgreSQL (RDS). My philosophy: start at the entry point and follow the path one hop at a time. Don't skip ahead and guess — that wastes time if wrong.

Debugging Diagram

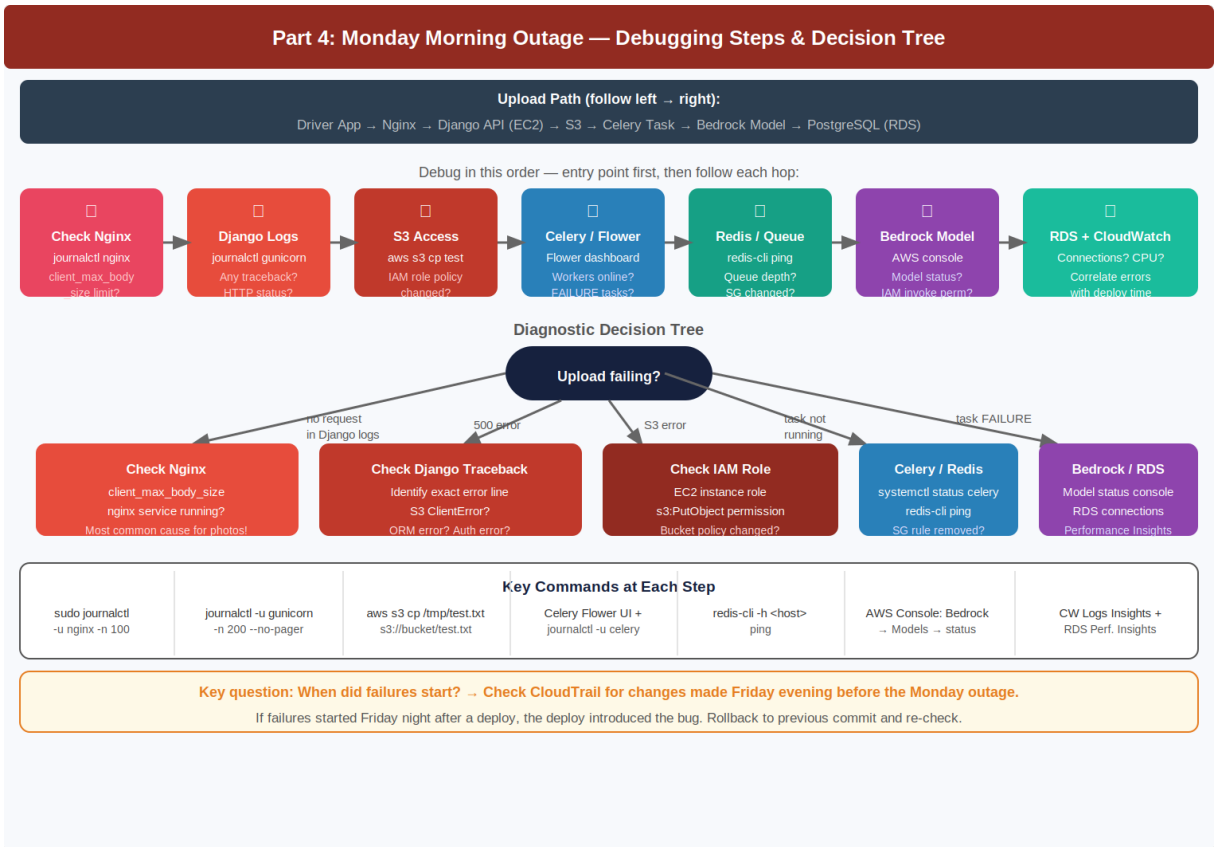

Figure 5: Debug order, decision tree, and common root causes by component

Step 1: Step 1: Check what error the driver app is seeing

Before touching the server, I check what the driver app actually reports: HTTP 4xx, 5xx, network timeout, or a silent failure? This immediately narrows the search space. A 413 means Nginx body size limit. A 500 means Django crashed. A timeout means Nginx/Gunicorn is not responding at all.

```
# Check Celery Flower or ask the driver to check the app error screen
# No server access needed yet
```

Why / what I'm looking for: Saves time by ruling out entire categories before SSHing.

Step 2: Step 2: Nginx logs (first server check)

SSH into EC2 and check Nginx first because it sits in front of Gunicorn and can reject requests before Django ever sees them. POD photos are 2–10MB. Nginx has a default client_max_body_size of 1MB.

```
sudo journalctl -u nginx -n 200 --no-pager
# or:
sudo tail -200 /var/log/nginx/error.log
# Check body size config:
grep -r 'client_max_body_size' /etc/nginx/
```

Why / what I'm looking for: This is the most common cause of photo upload failures. If Nginx was redeployed Friday evening, this config may have been reset to default.

Step 3: Step 3: Django/Gunicorn application logs

If Nginx logs are clean, the request is reaching Django. Check for Python tracebacks.

```
sudo journalctl -u gunicorn -n 300 --no-pager
# or:
sudo tail -300 /var/log/gunicorn/error.log
sudo tail -300 /var/log/django/app.log
```

Why / what I'm looking for: Look for: botocore.exceptions.ClientError (S3 problem), PermissionError, 500 Internal Server Error lines. The traceback will point to the exact failing line.

Step 4: Step 4: S3 permissions

If Django logs show an S3-related error (ClientError, AccessDenied), the EC2 instance role may have lost its S3 permission, or the bucket policy changed.

```
# Test S3 write directly from EC2:
aws s3 cp /tmp/test.txt s3://your-pod-bucket/debug-test.txt
# Check if it works. If not:
aws iam get-instance-profile --instance-profile-name your-profile
# Check CloudTrail for IAM changes made Friday:
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=PutRolePolicy \
  --start-time 2024-04-12T00:00:00
```

Why / what I'm looking for: AWS credentials rotation or IAM policy changes are common weekend maintenance tasks that cause Monday morning failures.

Step 5: Step 5: Celery worker and task queue

S3 upload succeeded but the Celery task (Bedrock validation) isn't running.

```
# Celery Flower dashboard (http://ec2-host:5555):
#   → Workers tab: are workers showing as Online?
#   → Tasks tab: filter by FAILURE or PENDING
#   → Queue depth: are tasks piling up unprocessed?

# Check worker logs:
sudo journalctl -u celery -n 300 --no-pager

# Test Redis connectivity from EC2:
redis-cli -h <elasticache-endpoint> -p 6379 ping
```

Why / what I'm looking for: Common Celery failures: worker OOM-killed (check CloudWatch MemoryUtilization), Redis unreachable (ElastiCache security group rule removed), or the worker is running but tasks are failing immediately.

Step 6: Step 6: Bedrock model

If Celery tasks show FAILURE state with a Bedrock-related exception.

```
# In the Celery task traceback, look for:
#   botocore.exceptions.ClientError: AccessDeniedException
#   ModelNotReadyException or ThrottlingException

# AWS Console: Bedrock → Models → check model availability
# CloudWatch metrics:
#   AWS/Bedrock/InvocationErrors
#   AWS/Bedrock/InvocationLatency

# Verify EC2 IAM role has bedrock:InvokeModel permission:
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::ACCOUNT:role/EC2Role \
  --action-names bedrock:InvokeModel
```

Why / what I'm looking for: Monday cold starts after a quiet weekend can cause brief Bedrock delays. More likely: the model ID hardcoded in settings was changed in a Friday deploy.

Step 7: Step 7: RDS connections and CloudWatch timeline

If Bedrock succeeds but results aren't saved, or to confirm the timeline of all findings.

```
# RDS Console: Performance Insights → DatabaseConnections graph
# Is the max_connections limit being hit?

# On EC2 — count current DB connections:
python manage.py dbshell
-- SELECT count(*) FROM pg_stat_activity;
-- SELECT max_conn FROM pg_settings WHERE name='max_connections';

# CloudWatch Logs Insights (run on Django log group):
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 100

# CloudTrail: any changes Friday night?
# EC2: CPUUtilization, NetworkIn spikes around 9 AM?
```

Why / what I'm looking for: CloudWatch gives me the exact timestamp when failures started. If failures started Friday 11 PM right after a deployment, that deployment introduced the bug. If they started at 9 AM Monday when users arrived, it's likely a scaling issue (connection pool, Celery worker count).

Summary

The most likely root cause for a Monday morning POD photo upload failure after a Friday deployment is **Nginx client_max_body_size reset to default (1MB)** — photos are 2–8MB and this is a common oversight when redeploying Nginx config. Second most likely: an IAM role change losing S3 write permission. Third: Celery worker failing to start after a restart, leaving tasks queued but unprocessed. Following the path step-by-step rather than guessing takes ~10 minutes and finds the cause reliably.

End of Submission — Happy to walk through any of these decisions in the follow-up call. GitHub Actions YAML files are in .github/workflows/ in the accompanying repository.